

HELWAN INTERNATIONAL TECHNOLOGY UNIVERSITY

Faculty of Engineering — Embedded Systems & IoT

SarcoOS

Elite Modular RTOS Framework for Teleoperation & Robotics

Comprehensive Technical Manual & Design Reference

Academic Year 2024 / 2025

Supervisor: Dr. Ahmed Suwaidan

Project Team

Ziad Mohamed Fathy
Abdelrahman Ashraf
Abdelrahman Ahmed
Abdelrahman Alaa
Ahmed Zaghloul
Ziad Hussein Mohamed Salah
Seif Ayman
Ammar Yasser
Sondos Mohamed

Chapter 1: Introduction & Project Overview

1.1 Project Background and Motivation

The rapid advancement of robotics and teleoperation systems has created an urgent demand for reliable, modular, and real-time capable firmware frameworks that can bridge the gap between high-level human intent and low-level electromechanical actuation. The SarcoOS project addresses this demand by engineering a comprehensive, production-quality robotic control ecosystem built upon the ESP32 microcontroller platform.

Modern teleoperated robotic systems must simultaneously handle sensor data acquisition, real-time closed-loop control, wireless communication, and human-machine interaction — all with deterministic timing guarantees and fault-tolerant behavior. The SarcoOS framework was conceived to unify these concerns under a single, well-architected codebase that is both academically rigorous and industrially practical.

1.2 Project Identity

Field	Details
Project Name	SarcoOS — Elite Modular RTOS Framework for Teleoperation & Robotics
University	Helwan International Technology University (HITU)
Supervisor	Dr. Ahmed Suwaidan (د. احمد سويدان)
Academic Year	2024 / 2025
Platform	ESP32 DevKitC-32E Raspberry Pi 4 Model B
Primary Language	Modern C++ (C++17) Python (ROS 2)
Framework	ESP-IDF v5.x FreeRTOS ROS 2 Humble

1.3 Project Team

Member	Arabic Name	Role
Ziad Mohamed Fathy	زياد محمد فتحي	Lead Embedded Software & IoT — Full Software Stack

Member	Arabic Name	Role
Abdelrahman Ashraf	عبد الرحمن اشرف	Hardware Integration & PCB Design
Abdelrahman Ahmed	عبد الرحمن احمد	Mechanical Design & CAD (SolidWorks)
Abdelrahman Alaa	عبد الرحمن علاء	ROS 2 & Simulation (Gazebo/URDF)
Ahmed Zaghloul	احمد زغلول	SLAM & Navigation Stack
Ziad Hussein Mohamed Salah	زياد حسين محمد صلاح	Teleoperation Suit Development
Seif Ayman	سيف ايمن	Motor Control & PID Tuning
Ammar Yasser	عمار ياسر	IIoT Dashboard & Node-RED HMI
Sondos Mohamed	سندس محمد	Testing, Validation & Documentation

1.4 System Concept: Two-Node Architecture

The SarcoOS ecosystem is divided into two primary hardware nodes that communicate wirelessly over a standard WiFi network using the MQTT protocol. This separation of concerns — input capture versus output actuation — is a fundamental architectural decision that provides resilience, modularity, and the ability to independently develop and test each subsystem.

- SUIT (Input Unit): A wearable teleoperation interface worn by a human operator. It captures joint angles via potentiometers, inertial orientation via MPU6050, and binary commands via push buttons. All data is published to an MQTT broker at 50 Hz.
- ROVER (Output Unit): A mobile robotic platform that subscribes to MQTT topics and translates incoming telemetry into physical motion via DC motors, servo actuators, and stepper motors, using closed-loop PID control where precision is required.

1.5 Design Philosophy

The SarcoOS codebase adheres to the following core design principles, which are reflected throughout every layer of the architecture:

- Strict Layering: Application logic never accesses hardware registers directly. All hardware interaction passes through the HAL (Hardware Abstraction Layer) or MCAL (Microcontroller Abstraction Layer).

- **Separation of Concerns:** Each driver class has a single, well-defined responsibility. A motor driver controls motion; a sensor driver reads data; a communication driver handles the network. They are not entangled.
- **Deterministic Timing:** FreeRTOS tasks use `vTaskDelayUntil()` for fixed-period loops, ensuring that control algorithms receive accurate time-step inputs regardless of variable execution time within the task body.
- **Fail-Safe Defaults:** All motor drivers initialize in a disabled, braked state. Emergency stop logic (via MQTT button topic with QoS 1) takes priority over all other commands.
- **Calibration-Aware Hardware:** ADC readings use factory eFuse calibration coefficients. IMU sensors are calibrated at startup with statistical averaging of 200 samples.

1.6 Document Structure

This manual is organized into the following chapters to provide a comprehensive understanding of the SarcoOS system from every dimension:

1. Chapter 1 — Introduction & Project Overview (this chapter)
2. Chapter 2 — Technology Stack: Rationale and Architecture
3. Chapter 3 — ESP32 Firmware: Hardware Abstraction Layer (HAL)
4. Chapter 4 — ESP32 Firmware: Microcontroller Abstraction Layer (MCAL)
5. Chapter 5 — Application Layer & FreeRTOS Task Architecture
6. Chapter 6 — Custom PCB Hardware Design
7. Chapter 7 — Mechanical & 3D Design
8. Chapter 8 — Teleoperation Wearable Suit
9. Chapter 9 — ROS 2, Simulation & SLAM
10. Chapter 10 — Performance, Safety & System Validation

Chapter 2: Technology Stack — Rationale & Architecture

2.1 Programming Language: Modern C++ (C++17)

The decision to implement the SarcoOS firmware in Modern C++ (specifically the C++17 standard) was not arbitrary. It was the result of a careful evaluation of the language's strengths against the demanding requirements of a real-time robotic control system operating on a resource-constrained microcontroller.

2.1.1 Object-Oriented Modularity

C++ classes provide a natural mapping from the physical hardware hierarchy to the software architecture. Each physical component — a motor driver IC, an IMU sensor, a wireless module — becomes a C++ class with a well-defined interface. This encapsulation ensures that the internal complexity of driving a BTS7960 H-bridge via LEDC PWM channels is hidden behind a simple `forward(speed)` method, dramatically reducing cognitive load for application developers.

2.1.2 Zero-Cost Abstractions

C++ templates and inline functions allow the construction of high-level abstractions — such as the `NVS::read<T>()` template — without incurring runtime overhead. The compiler resolves template instantiations at compile time, generating code that is equivalent in performance to hand-written C, while providing full type safety.

2.1.3 RAI and Resource Safety

Resource Acquisition Is Initialization (RAII) patterns ensure that hardware peripherals are properly initialized in constructors and released in destructors. This eliminates entire categories of bugs common in embedded C code, such as forgotten peripheral de-initialization or double-free errors on timer channels.

2.1.4 Why Not Arduino?

Why Not Arduino?

While Arduino provides a rapid prototyping environment, it abstracts too aggressively — hiding FreeRTOS, ESP-IDF APIs, and hardware peripherals behind high-level wrappers that are neither deterministic nor efficient enough for production robotic control. SarcoOS requires direct access to the LEDC peripheral for precise PWM, the ADC calibration API for accurate voltage readings, and the UART driver for GPS parsing. Arduino's HAL does not expose these at the required level of control.

2.2 Framework: ESP-IDF (Espressif IoT Development Framework)

ESP-IDF is the official development framework for the ESP32 family, maintained by Espressif Systems. It provides a mature, production-quality environment that gives direct access to all hardware peripherals without the performance penalties of higher-level abstraction frameworks.

2.2.1 Key Advantages of ESP-IDF

- Direct peripheral access: LEDC, ADC, I2C, SPI, UART, RMT, and GPIO APIs expose the full capability of the ESP32 silicon.
- Integrated FreeRTOS: FreeRTOS is compiled as part of the ESP-IDF, not as an external library, ensuring tight integration between the RTOS scheduler and the hardware interrupt system.
- Factory Calibration Support: ESP-IDF exposes the `adc_cali_line_fitting` API for applying factory-stored eFuse calibration coefficients to ADC readings, reducing voltage error from $\pm 150\text{mV}$ to $\pm 30\text{mV}$.
- Production-Grade Networking: The lwIP TCP/IP stack, integrated WiFi driver, and `esp_mqtt_client` library provide enterprise-grade networking capabilities directly on the microcontroller.
- CMake Build System: ESP-IDF uses CMake natively, enabling reproducible builds, incremental compilation, and straightforward integration with CI/CD pipelines.

2.3 Build System: CMake

CMake serves as the build automation system for the SarcoOS project. The decision to use CMake — rather than makefiles or IDE-specific build systems — was driven by the need for cross-platform reproducibility and the ESP-IDF framework's native requirement for it.

The CMake configuration defines component dependencies, compile-time flags (such as C++17 standard enforcement), linker scripts, and partition table configurations. The modular component system allows each driver (`bts7960`, `mpu6050`, `pid`, etc.) to be compiled independently, reducing incremental build times during iterative development.

```
# Example CMakeLists.txt for the BTS7960 driver component
idf_component_register(
    SRCS "bts7960.cpp"
    INCLUDE_DIRS "."
    REQUIRES driver ledc
)
target_compile_options(${COMPONENT_LIB} PRIVATE -std=c++17 -Wall -Wextra)
```

2.4 Real-Time Operating System: FreeRTOS

FreeRTOS is the foundational scheduling layer of SarcoOS. Its selection over bare-metal programming was driven by the inherent concurrent nature of the robotic control problem: the

system must simultaneously poll sensors, run PID control loops, publish MQTT data, and respond to incoming command callbacks — all with bounded latency guarantees.

2.4.1 Key FreeRTOS Features Used

Feature	Usage in SarcoOS	Benefit
Tasks (xTaskCreate)	input_task, output_task, pos_control_task	Concurrent execution of independent concerns
vTaskDelayUntil()	Fixed-period sensor polling (50 Hz)	Deterministic timing — compensates for variable task execution time
Task Priorities	PID tasks at priority 7, I/O at priority 5	Critical control loops preempt lower-priority communication tasks
Semaphores	MQTT callback to control task signaling	Thread-safe data sharing between ISR context and task context
FreeRTOS Heap	Stack allocation for each task	Predictable memory layout; stack overflow detection available

2.4.2 Deterministic Control Loop Timing

The most critical aspect of FreeRTOS usage in SarcoOS is the use of vTaskDelayUntil() instead of vTaskDelay() for control loops. The distinction is fundamental:

```
// INCORRECT: vTaskDelay causes period to drift under load
while(true) {
    pid_update();           // takes variable time
    vTaskDelay(pdMS_TO_TICKS(20)); // 20ms AFTER task completes
}

// CORRECT: vTaskDelayUntil maintains absolute 50Hz period
TickType_t last = xTaskGetTickCount();
while(true) {
    pid_update();           // takes variable time
    vTaskDelayUntil(&last, pdMS_TO_TICKS(20)); // 20ms FROM last wake
}
```

With vTaskDelayUntil(), the PID controller always receives a consistent dt = 0.020 seconds, which is mathematically required for accurate integral accumulation and derivative computation.

2.5 Layered Architecture: HAL/MCAL

The firmware architecture is organized into three strict tiers, each building upon the one below it. This layered approach is directly inspired by the AUTOSAR software architecture model used in automotive embedded systems, adapted for the ESP32 platform.

Tier	Layer Name	Responsibilities	Examples
Tier 3 (Top)	Application Layer	Mission-specific business logic, task orchestration, system-level behavior	main.cpp, esp32_inputs.cpp, esp32_outputs.cpp, pot_pos_control.cpp
Tier 2 (Mid)	Hardware Abstraction Layer (HAL)	Device drivers, peripheral interfaces, signal conditioning, closed-loop control	BTS7960, Servo, MPU6050, Potentiometer, PID, Button, LED, Ultrasonic
Tier 1 (Base)	Microcontroller Abstraction Layer (MCAL)	Silicon-level abstractions for connectivity, storage, and serial communication	WiFiManager, MQTTClient, NVS, uart

The key rule of this architecture is that each tier may only call downward into the tier below it. Application code calls HAL APIs; HAL drivers call MCAL APIs and ESP-IDF peripheral APIs; MCAL calls ESP-IDF directly. This ensures a clean dependency graph and prevents circular dependencies.

2.6 IIoT Stack: MQTT + Node-RED + Mosquitto

2.6.1 MQTT Protocol

MQTT (Message Queuing Telemetry Transport) was selected as the communication protocol for the Suit-to-Rover link because of its lightweight publish/subscribe model, which is ideally suited for the asynchronous, real-time telemetry use case of SarcoOS.

- Payload overhead: MQTT has a fixed header of only 2 bytes, compared to HTTP's hundreds of bytes per request. This is critical for the 50 Hz data stream, where low latency is mandatory.
- Topic-based routing: The broker routes messages based on topic strings (/sarcoos/pot/1, /sarcoos/btn/1, etc.), allowing the Rover to subscribe only to relevant data streams.
- QoS levels: QoS 0 (fire-and-forget) is used for high-frequency potentiometer data to minimize latency. QoS 1 (at-least-once) is used for emergency stop commands to guarantee delivery.

2.6.2 Mosquitto Broker

The Eclipse Mosquitto broker is deployed on a network-accessible server (or on the Raspberry Pi 4). It serves as the central message router for the entire SarcoOS ecosystem, receiving publications from the Suit and routing them to all subscribers including the Rover and the Node-RED dashboard simultaneously.

2.6.3 Node-RED HMI Dashboard

Node-RED provides the Human-Machine Interface (HMI) for the SarcoOS system. It subscribes to all MQTT telemetry topics and renders real-time gauges, graphs, and control widgets for operator situational awareness. The Node-RED flow is configured to display potentiometer angles, IMU orientation, motor status, and system health indicators.

Chapter 3: ESP32 Firmware — Hardware Abstraction Layer (HAL)

The Hardware Abstraction Layer (HAL) forms the core of the SarcoOS firmware. It comprises twelve driver classes that collectively interface with every electromechanical component in the robotic ecosystem. Each driver encapsulates pin configuration, communication protocols, interrupt service routines, and signal conditioning, exposing a clean C++ API to the application layer above.

3.1 BTS7960 — High-Power H-Bridge Motor Driver

3.1.1 Hardware Background

The BTS7960 is a half-bridge IC manufactured by Infineon Technologies, rated for continuous drain currents up to 43A and peak currents of 70A. A single H-bridge motor channel in SarcoOS uses two BTS7960 ICs in a half-bridge configuration: one handles the HIGH-side switch (current flow into the motor), the other handles the LOW-side switch (current flow out of the motor). This arrangement enables full bidirectional motor control with regenerative braking capability.

The motor direction is controlled by which half-bridge is driven with PWM: asserting R_PWM drives current in the forward direction, while asserting L_PWM drives current in reverse. The R_EN and L_EN enable pins must be driven HIGH before any PWM signal is applied; leaving them LOW places the driver in a low-power tri-state mode that disconnects the motor terminals.

3.1.2 PWM Generation: LEDC Peripheral

The ESP32 LEDC (LED Control) peripheral, despite its name, is a versatile PWM generator used throughout SarcoOS for motor control. It provides 8 independent PWM channels with 20-bit resolution timers. The BTS7960 driver configures the LEDC with a 20 kHz frequency — above the human audible range of 20 Hz to 20 kHz — to eliminate motor whine. At 20 kHz with 12-bit resolution, the duty cycle can be set in 4096 discrete steps, providing fine-grained speed control.

```
// BTS7960 Constructor - LEDC Configuration
BTS7960::BTS7960(gpio_num_t r_en, gpio_num_t l_en,
                 gpio_num_t r_pwm, gpio_num_t l_pwm, uint8_t freq_hz) {
    // Configure enable pins as outputs
    gpio_set_direction(r_en, GPIO_MODE_OUTPUT);
    gpio_set_direction(l_en, GPIO_MODE_OUTPUT);
    // Configure LEDC timer
    ledc_timer_config_t timer = {
        .speed_mode = LEDC_LOW_SPEED_MODE,
        .duty_resolution = LEDC_TIMER_12_BIT,
        .timer_num = LEDC_TIMER_0,
        .freq_hz = freq_hz,
        .clk_cfg = LEDC_AUTO_CLK
    };
};
```

```
    ledc_timer_config(&timer);  
    // Bind R_PWM and L_PWM to LEDC channels  
    // ... channel config for both pins ...  
}
```

3.1.3 API Reference

Method	Parameters	Returns	Description
BTS7960(r_en, l_en, r_pwm, l_pwm, freq)	gpio_num_t x4, uint8_t	—	Constructor. Configures LEDC timer and channels. Default 20 kHz is above audible range.
setSpeed(speed, dir)	float 0.0-1.0, MOTOR_DIRECTION	void	Sets normalized PWM duty cycle. FORWARD activates r_pwm; BACKWARD activates l_pwm.
forward(speed)	float 0.0-1.0	void	Convenience wrapper for FORWARD direction.
backward(speed)	float 0.0-1.0	void	Convenience wrapper for BACKWARD direction.
stop()	—	void	Sets both PWM channels to zero. Motor coasts.
brake()	—	void	Activates both enable and PWM at max simultaneously for regenerative braking.
enable()	—	void	Asserts R_EN and L_EN HIGH. Driver becomes active.
disable()	—	void	De-asserts enable pins. Driver enters tri-state low-power mode.

Safety Rule

Never call `setSpeed()` with the driver disabled. Always call `enable()` first to prevent undefined behavior from floating enable pins. The BTS7960 ICs have internal protection circuits, but floating enables can cause spurious current spikes.

3.2 BTS7960_Pot_Position — Closed-Loop Position Controller

3.2.1 Concept: High-Torque Servo Replacement

The `BTS7960_Pot_Position` class is the most architecturally sophisticated driver in the SarcoOS HAL. It transforms a standard DC motor with a potentiometer position sensor into a closed-loop servo actuator capable of holding and moving to precise angular positions. This architecture is used in SarcoOS to drive heavy robotic arm joints where standard RC servos — typically limited to 20-35 kg-cm torque — are insufficient.

The class is a composite object: it contains references to a BTS7960 motor driver and a Potentiometer, and embeds a PID controller instance. The `update(dt)` method, called at 50 Hz from a dedicated FreeRTOS task, reads the current potentiometer angle, computes the PID error relative to the commanded setpoint, and drives the motor in the corrective direction.

3.2.2 Control Loop Mathematics

The PID control algorithm executed within `update(dt)` follows the standard discrete-time implementation:

```
float error = setpoint - current_angle;           // Position error (degrees)
float P = kp * error;                             // Proportional: immediate correction
float I += ki * error * dt;                       // Integral: accumulated offset
elimination
float I = clamp(I, output_min, output_max);       // Anti-windup clamping
float D = kd * (error - prev_error) / dt;         // Derivative: damping oscillation
float output = clamp(P + I + D, -1.0, 1.0);       // Motor command: -1.0 to +1.0
```

The output value directly maps to motor duty cycle and direction: positive output calls `motor.forward(output)`, negative output calls `motor.backward(-output)`. When the absolute error falls below the configured tolerance (default 2.0 degrees), the motor is stopped and the `isAtTarget()` flag is set.

3.2.3 API Reference

Method	Parameters	Returns	Description
BTS7960_Pot_Position (motor, pot, kp, ki, kd)	BTS7960&, Potentiometer&, float x3	—	Constructor. Initializes composite object with PID gains.
setAngle(target_deg)	float	void	Commands target joint angle in degrees. Begins closed-loop PID.
getAngle()	—	float	Reads current pot angle without invoking PID computation.
update(dt)	float (seconds)	void	Core control loop tick. Must be called at fixed intervals.
isAtTarget()	—	bool	Returns true when error < tolerance threshold (default 2.0 deg).

3.3 Potentiometer — Advanced ADC Driver

3.3.1 ADC Architecture on ESP32

The ESP32 features two Successive Approximation Register (SAR) ADC units: ADC1 (8 channels on GPIO32-GPIO39) and ADC2 (10 channels, shared with WiFi). SarcoOS exclusively uses ADC1 because ADC2 is unavailable when the WiFi radio is active — a critical constraint for the MQTT-connected Rover. All potentiometer channels are therefore mapped to GPIO32-GPIO39.

3.3.2 Calibration Scheme

The raw ESP32 ADC suffers from significant non-linearity, particularly at the extremes of its voltage range. Without calibration, voltage readings can have errors of $\pm 150\text{mV}$. The SarcoOS Potentiometer driver applies ESP-IDF's `adc_cali_line_fitting` calibration scheme, which uses a third-order polynomial correction with coefficients derived from factory characterization and stored in the ESP32's eFuse memory. After calibration, voltage reading errors are reduced to $\pm 30\text{mV}$.

```
// Calibration initialization in Potentiometer constructor
adc_cali_line_fitting_config_t cali_config = {
    .unit_id      = ADC_UNIT_1,
    .atten        = ADC_ATTEN_DB_11,    // Full 0-3.3V input range
    .bitwidth     = ADC_BITWIDTH_12,    // 12-bit = 4096 steps
    .default_vref = 1100                // mV fallback if eFuse unavailable
};
```

```
adc_cali_create_scheme_line_fitting(&cali_config, &cali_handle);
```

3.3.3 Attenuation Selection

The ADC_ATTEN_DB_11 attenuation setting is selected for all potentiometer channels in SarcoOS. This setting expands the ADC input voltage range from the default 0-1.1V to the full 0-3.3V range, allowing the potentiometer's voltage divider output (which swings across the full 3.3V supply rail) to be directly connected to the ADC pin without any external voltage scaling circuitry.

3.3.4 API Reference

Method	Parameters	Returns	Description
Potentiometer(channel, atten, max_angle)	adc_channel_t, adc_atten_t, int	—	Constructor. Configures ADC1 channel with calibration.
readRaw()	—	int (0-4095)	Returns raw 12-bit ADC conversion without processing.
readVoltage()	—	float (mV)	Returns calibrated voltage using adc_cali_line_fitting.
readAngle()	—	float (deg)	Maps calibrated voltage to angular position (0 to max_angle).
readPercentage()	—	float (0-100)	Expresses position as percentage of full mechanical range.
readMapped(min, max)	float, float	float	Maps ADC reading to arbitrary user-defined numeric range.
readAveraged(samples)	int (1-64)	float (deg)	Performs N reads, returns mean in degrees. Reduces EMI noise.
setMaxAngle(angle)	int (degrees)	void	Reconfigures full-scale angle without reinstantiation.

3.4 MPU6050 — 6-DOF Inertial Measurement Unit

3.4.1 Sensor Overview

The InvenSense MPU-6050 is a 6-axis MEMS sensor that combines a 3-axis accelerometer and a 3-axis gyroscope in a single package, communicating over I2C. In SarcoOS it serves two purposes: on the Suit, it measures the operator's arm orientation to provide IMU-based control inputs; on the Rover, its data can be used for stabilization feedback.

3.4.2 I2C Communication

The MPU6050 communicates at up to 400 kHz (Fast-mode I2C). The SarcoOS driver configures ESP32's I2C peripheral (I2C_NUM_0) at 400 kHz with SDA on GPIO21 and SCL on GPIO22. The I2C address is configurable: 0x68 when the AD0 pin is tied LOW, and 0x69 when AD0 is tied HIGH, allowing two MPU6050 devices on the same I2C bus.

3.4.3 Complementary Filter for Orientation Fusion

Raw gyroscope data provides fast, accurate short-term angle tracking but accumulates drift over time. Raw accelerometer data is noisy but provides an absolute reference (gravity vector) that does not drift. The complementary filter combines both sources optimally:

```
// Complementary filter: alpha = 0.96 (96% gyro, 4% accelerometer)
const float ALPHA = 0.96f;
// Accelerometer angles from gravity vector
accel_angle_x = atan2(ay, az) * RAD_TO_DEG;
accel_angle_y = atan2(-ax, sqrt(ay*ay + az*az)) * RAD_TO_DEG;
// Fused angles
roll  = ALPHA * (roll  + gx * dt) + (1.0f - ALPHA) * accel_angle_x;
pitch = ALPHA * (pitch + gy * dt) + (1.0f - ALPHA) * accel_angle_y;
yaw  += gz * dt; // Gyro-only (no absolute yaw reference without magnetometer)
```

The alpha coefficient of 0.96 represents a design trade-off: higher alpha values favor the gyroscope (faster response, more drift), while lower values favor the accelerometer (more noise, but drift-free). The value 0.96 was empirically tuned for the expected motion dynamics of the teleoperation suit.

3.4.4 API Reference

Method	Parameters	Returns	Description
MPU6050(port, sda, scl, addr, freq)	i2c_port_t, gpio x2, uint8_t, uint32_t	—	Constructor. Initializes I2C bus and wakes sensor from sleep mode.
readAccel(ax, ay, az)	float& x3 (output)	esp_err_t	Reads 3-axis acceleration in g units (±2g default scale).

Method	Parameters	Returns	Description
readGyro(gx, gy, gz)	float& x3 (output)	esp_err_t	Reads 3-axis angular rate in deg/s (± 250 deg/s default scale).
updateAngles(dt)	float (seconds)	void	Computes fused Roll, Pitch, Yaw using complementary filter.
getRoll()	—	float	Returns current Roll angle in degrees (rotation about X-axis).
getPitch()	—	float	Returns current Pitch angle in degrees (rotation about Y-axis).
getYaw()	—	float	Returns Yaw angle. Note: drifts over time without magnetometer.
calibrate(samples)	int (default 200)	void	Averages N readings at rest to compute bias offsets.

3.5 Servo — Intelligent PWM Servo Controller

3.5.1 PWM Servo Standard

RC servos are controlled by a 50 Hz PWM signal where the pulse width encodes the command. Standard servos follow the 1000-2000 μ s convention: 1000 μ s commands full counter-clockwise, 1500 μ s commands center, and 2000 μ s commands full clockwise. SarcoOS extends this to the 500-2500 μ s range to accommodate high-range servos and continuous rotation servos.

The LEDC peripheral generates the 50 Hz signal. With a 20-bit timer at 80 MHz clock, each tick represents 1/80,000,000 seconds. A 50 Hz period (20 ms) corresponds to 1,600,000 ticks. A 1 μ s pulse width corresponds to 80 ticks. The setAngle(deg) method performs this conversion:

```
// Pulse width calculation for setAngle()
float pulse_us = 500.0f + (angle / 180.0f) * 2000.0f; // 500-2500  $\mu$ s range
uint32_t duty = (uint32_t)((pulse_us / 20000.0f) * (1 << 16)); // LEDC 16-bit
ledc_set_duty(LEDC_LOW_SPEED_MODE, channel, duty);
ledc_update_duty(LEDC_LOW_SPEED_MODE, channel);
```

3.5.2 Operating Modes

Mode	Constant	Behavior	Use Case
Position Control	STANDARD_180	setAngle(0-180) maps linearly to 500-2500 μ s	Arm joints, pan/tilt mechanisms
Continuous Rotation	CONTINUOUS_360	setSpeed() controls rotation rate and direction	Wheel drive, conveyor mechanisms

3.6 PID — Industrial-Grade Controller

3.6.1 PID Theory

The Proportional-Integral-Derivative (PID) controller is the most widely deployed feedback control algorithm in industry. SarcoOS implements a discrete-time PID with three industrial-grade features: integral anti-windup, output clamping, and derivative-on-measurement (to prevent derivative kick on setpoint changes).

The continuous-time PID transfer function is:

$$u(t) = K_p * e(t) + K_i * \text{integral}(e(t)) \, dt + K_d * d(e(t))/dt$$

In discrete-time (sampled at period $T = dt$), this becomes:

```
e[k] = setpoint - measurement[k]
P[k] = Kp * e[k]
I[k] = I[k-1] + Ki * e[k] * dt           // Euler integration
D[k] = Kd * (e[k] - e[k-1]) / dt       // Backward difference
u[k] = clamp(P[k] + I[k] + D[k], -1, 1)
```

3.6.2 Integral Anti-Windup

Integral windup occurs when the system output is physically limited (e.g., a motor at maximum duty cycle) but the integral term continues accumulating error. When the obstruction is removed, the accumulated integral causes a large, oscillatory overshoot. SarcoOS prevents this by clamping the integral accumulator to the same bounds as the output:

```
// Anti-windup: clamp integral to output bounds
integral += ki * error * dt;
integral = std::clamp(integral, output_min, output_max); // KEY LINE
```

3.6.3 PID Tuning Guidelines

PID gains must be tuned for each specific motor-load combination. The following procedure is recommended for SarcoOS joint controllers:

11. Set $K_i = 0$, $K_d = 0$. Increase K_p until the system reaches the setpoint quickly but begins to oscillate slightly.
12. Increase K_d to damp the oscillation. The system should now converge without ringing.
13. Increase K_i slowly to eliminate any steady-state error. Monitor for windup using `getIntegral()`.

3.6.4 API Reference

Method	Parameters	Returns	Description
<code>PID(kp, ki, kd, min, max)</code>	float x5	—	Constructor. Default output bounds [-1.0, +1.0].
<code>updatePID(setpoint, meas, dt)</code>	float, float, float	float	Core computation. Returns clamped PID output.
<code>reset()</code>	—	void	Zeroes integral and previous error. Call on setpoint changes.
<code>setGains(kp, ki, kd)</code>	float x3	void	Runtime gain tuning without reinstantiation.
<code>setOutputLimits(min, max)</code>	float, float	void	Reconfigures output clamping range.
<code>getProportional()</code>	—	float	Returns last P-term (diagnostic).
<code>getIntegral()</code>	—	float	Returns I-term accumulator (diagnostic/anti-windup monitor).
<code>getDerivative()</code>	—	float	Returns last D-term (diagnostic).

3.7 Additional HAL Drivers

3.7.1 BTS7960_Position — Encoder-Based Controller

A legacy closed-loop position controller that uses a quadrature encoder for position feedback instead of a potentiometer. The encoder ISR counts pulses on each rising/falling edge of Encoder Channel A, with Channel B read inside the ISR to determine direction. The ISR function is

decorated with `IRAM_ATTR` to place it in Internal RAM, ensuring execution even during flash cache misses.

```
// Encoder ISR - IRAM required for cache-miss immunity
static void IRAM_ATTR encoder_isr_handler(void* arg) {
    BTS7960_Position* self = (BTS7960_Position*)arg;
    if (gpio_get_level(self->enc_b_pin)) {
        self->pulse_count++;
    } else {
        self->pulse_count--;
    }
}
```

3.7.2 TCRT5000 — Reflective IR Sensor

The TCRT5000 infrared reflectance sensor is used for line-following applications. It operates in two modes: digital threshold mode (returns 0/1 based on a hardware comparator) and analog raw mode (returns ADC value proportional to IR reflectance). The SarcoOS driver supports both modes simultaneously through the `tcrs_sensor` class.

3.7.3 HC-SR04 — Ultrasonic Distance Sensor

The HC-SR04 driver measures distance by timing the echo pulse from an ultrasonic burst. A 10 μ s trigger pulse is generated on the TRIG pin; the duration of the returning HIGH pulse on the ECHO pin is measured using `esp_timer_get_time()` for microsecond precision. Distance in centimeters is derived as: $\text{distance} = \text{echo_pulse_}\mu\text{s} / 58.0$.

3.7.4 Button — Debounced Digital Input

The Button driver provides debounced GPIO input reading. Software debouncing implements a 20 ms lockout window using `esp_timer` to reject contact bounce. The driver supports both active-HIGH and active-LOW (pull-up) configurations, with logical inversion applied automatically for pull-up mode so that pressing the button always returns logical 1.

3.7.5 LED — Digital Output with Timed Toggle

The LED driver provides simple GPIO output control with `ON()`, `OFF()`, and `TOGGLE(period_ms)` methods. The `TOGGLE` method uses `vTaskDelay` internally to create a continuous blink at the specified period, making it suitable for status indication without blocking the calling task (should be called from a dedicated status task).

Chapter 4: Microcontroller Abstraction Layer (MCAL)

The MCAL provides a clean, object-oriented C++ interface to the ESP32's silicon-level capabilities for connectivity, persistent storage, and serial communication. By abstracting ESP-IDF's C APIs into C++ classes, the MCAL enables application and HAL code to be written and tested independently of the specific ESP-IDF API version.

4.1 WiFiManager — Station Mode Connection Manager

4.1.1 Architecture

The WiFiManager class manages the complete lifecycle of a WiFi Station (STA) mode connection. It registers event handlers with the ESP-IDF default event loop for WIFI_EVENT and IP_EVENT events, implements automatic reconnection with configurable retry limits, and exposes callback hooks that higher layers use to trigger dependent operations (such as initiating MQTT connection after IP address acquisition).

4.1.2 Event-Driven Connection Flow

The connection process follows an asynchronous, event-driven pattern:

14. `wifi.connect()` is called, which calls `esp_wifi_start()` — non-blocking.
15. The ESP-IDF WiFi driver internally scans for the target SSID.
16. On successful association, `WIFI_EVENT_STA_CONNECTED` fires.
17. The lwIP DHCP client requests an IP address.
18. On IP acquisition, `IP_EVENT_STA_GOT_IP` fires.
19. WiFiManager invokes the `on_connected(ip)` callback registered by the application.
20. The application's `on_connected` callback calls `mqtt.connect()` — MQTT is now safe to start.

```
// Application layer usage - MQTT starts only after WiFi is confirmed
WiFiManager wifi("SarcoOS_Network", "securepassword");
MQTTClient mqtt("mqtt://192.168.1.100:1883");
wifi.on_connected([&mqtt](std::string ip) {
    ESP_LOGI(TAG, "IP assigned: %s - starting MQTT", ip.c_str());
    mqtt.connect();
});
wifi.on_disconnected([&mqtt]() {
    ESP_LOGW(TAG, "WiFi lost - pausing publish operations");
});
wifi.connect(); // Initiates non-blocking connection
```

4.1.3 API Reference

Method	Parameters	Returns	Description
WiFiManager(ssid, pw, retries)	const char* x2, uint8_t	—	Constructor. Registers WIFI_EVENT and IP_EVENT handlers.
connect()	—	esp_err_t	Initiates WiFi connection. Non-blocking; signals via on_connected.
disconnect()	—	void	Gracefully disconnects from AP. Triggers on_disconnected.
isConnected()	—	bool	Returns connection state from internal flag (no network round-trip).
on_connected(cb)	callback(ip)	void	User callback invoked when IP is assigned. Triggers MQTT connect.
on_disconnected(cb)	callback()	void	User callback invoked on disconnect. Pause MQTT operations.
getIP()	—	std::string	Returns last assigned IPv4 address as dotted-decimal string.

4.2 MQTTClient — MQTT 3.1.1 Pub/Sub Wrapper

4.2.1 Protocol Overview

MQTT (Message Queuing Telemetry Transport) version 3.1.1 is the communication backbone of SarcoOS. The MQTTClient class wraps ESP-IDF's esp_mqtt_client with a clean C++ interface that supports per-topic callback routing — a significant ergonomic improvement over the raw ESP-IDF API, which uses a single global event handler.

4.2.2 Per-Topic Callback Routing

The on_topic(topic, callback) method registers a dedicated handler for a specific topic string. Internally, the MQTTClient maintains a std::map from topic strings to handler functions. When an MQTT_EVENT_DATA event fires, the client looks up the incoming topic in the map and invokes the corresponding handler, or falls back to the global on_message handler for unregistered topics.

```
// Per-topic callback routing example
```

```
mqtt.on_topic("/sarcoos/pot/1", [&motor1](std::string data) {
    float angle = std::stof(data);           // Parse float from MQTT payload
    float speed = (angle / 135.0f) - 1.0f;    // Map 0-270° to -1.0 to +1.0
    if      (speed > 0) motor1.forward(speed);
    else if (speed < 0) motor1.backward(-speed);
    else      motor1.stop();
});
mqtt.on_topic("/sarcoos/btn/1", [&motor1](std::string data) {
    if (data == "1") motor1.brake(); // Emergency stop
});
mqtt.subscribe("/sarcoos/#"); // Wildcard subscription for all SarcoOS topics
```

4.2.3 QoS Strategy

SarcoOS uses a two-tier QoS strategy to balance real-time performance with reliability:

- QoS 0 (Fire-and-Forget): Used for high-frequency potentiometer angles (/sarcoos/pot/*) and IMU data (/sarcoos/imu/*). Dropped packets are acceptable — the next update will arrive within 20 ms. QoS 1 overhead would add unnecessary latency.
- QoS 1 (At-Least-Once): Used for all button commands (/sarcoos/btn/*), especially the emergency stop. Reliable delivery is mandatory for safety-critical commands. The broker stores QoS 1 messages for delivery to reconnecting clients.

4.3 NVS — Template-Based Non-Volatile Storage

4.3.1 Purpose

The ESP32's Non-Volatile Storage (NVS) partition provides a key-value store that persists across power cycles. SarcoOS uses NVS to store PID gain parameters, ADC calibration offsets, WiFi credentials, and system configuration flags that should survive resets. The NVS class provides a type-safe C++ wrapper using function templates that eliminate the manual type-checking required by the raw ESP-IDF nvs API.

```
NVS config("sarcoos_cfg");
// Persist PID gains for motor joint 1
config.write<float>("kp_motor1", 3.5f);
config.write<float>("ki_motor1", 0.05f);
config.write<float>("kd_motor1", 0.20f);
config.commit(); // Flush to flash
// Read back on next power cycle
float kp = config.read<float>("kp_motor1", 1.0f); // Default 1.0 if not found
```

4.3.2 Type Safety

The template parameters enforce type consistency between write and read operations. Attempting to read a float key as an int will return the default value and log a warning, preventing the subtle

bugs that occur when raw NVS type codes (NVS_TYPE_U32, NVS_TYPE_BLOB, etc.) are mixed up.

4.4 UART — Device-Structured Serial Communication

4.4.1 Usage in SarcoOS

The UART MCAL component is used primarily for GPS communication (Neo-6M module at 9600 baud on UART2) and debug logging (UART0). It abstracts ESP-IDF's `uart_driver_install()` and associated configuration calls behind a struct-based initialization pattern that mirrors the device-tree descriptor model used in Linux kernel drivers.

```
// UART device configuration for GPS
uart_device_t gps_uart = {
    .port      = UART_NUM_2,
    .baud_rate = 9600,
    .parity     = UART_PARITY_DISABLE,
    .stop_bits  = UART_STOP_BITS_1,
    .tx_pin     = GPIO_NUM_17,
    .rx_pin     = GPIO_NUM_16,
    .rx_buf_size = 1024
};
uart_gps(gps_uart);
// Read NMEA sentence
uint8_t buf[128];
int len = gps.read(buf, sizeof(buf), 100); // 100ms timeout
```

4.4.2 NMEA GPS Parsing

The Neo-6M GPS module transmits NMEA 0183 protocol sentences over UART. The `$GPGGA` sentence contains the primary fix data including latitude, longitude, altitude, and fix quality. The SarcoOS application layer parses these sentences by scanning for the sentence type identifier, then tokenizing the comma-delimited fields.

Chapter 5: Application Layer & FreeRTOS Task Architecture

5.1 System Initialization Sequence

The `app_main()` function in `main.cpp` is the ESP-IDF application entry point, analogous to `main()` in standard C. It executes a strictly ordered initialization sequence before spawning FreeRTOS tasks and returning control to the RTOS scheduler.

```
extern "C" void app_main(void) {
    // Step 1: Initialize NVS flash partition
    esp_err_t ret = nvs_flash_init();
    if (ret == ESP_ERR_NVS_NO_FREE_PAGES ||
        ret == ESP_ERR_NVS_NEW_VERSION_FOUND) {
        nvs_flash_erase();
        nvs_flash_init();
    }
    // Step 2: Initialize event loop (required for WiFi/MQTT)
    esp_event_loop_create_default();
    // Step 3: Construct MCAL objects
    WiFiManager wifi(SSID, PASSWORD);
    MQTTClient mqtt(BROKER_URL);
    // Step 4: Register callbacks
    wifi.on_connected([&mqtt](std::string ip) { mqtt.connect(); });
    // Step 5: Start WiFi
    wifi.connect();
    // Step 6: Spawn application tasks
    xTaskCreate(input_task, "inputs", 4096, &mqtt, 5, NULL);
    xTaskCreate(output_task, "outputs", 4096, &mqtt, 5, NULL);
    // app_main returns - FreeRTOS scheduler takes over
}
```

5.2 Task Architecture

Task	Priority	Stack	Period	Description
input_task (Suit)	5	4096 B	20 ms (50 Hz)	Polls all sensors and publishes to MQTT topics
output_task (Rover)	5	4096 B	Event-driven	Subscribes to MQTT; actuates motors via callbacks

Task	Priority	Stack	Period	Description
pos_control_task	7	2048 B	20 ms (50 Hz)	PID position control loop for joint actuators
MQTT event loop	Internal	Internal	Event-driven	Managed by esp_mqtt_client internally
WiFi event loop	Internal	Internal	Event-driven	Managed by ESP-IDF netif; handles DHCP/reconnect

5.3 Suit Input Pipeline (esp32_inputs.cpp)

The input_task runs on the Suit ESP32. At 50 Hz, it reads all hardware sensors and publishes their values to discrete MQTT topics. The 50 Hz sample rate was chosen to comfortably capture human motion (bandwidth approximately 5-10 Hz) with a factor of 5 oversampling for noise immunity.

Input Source	MQTT Topic	Payload	QoS
Potentiometer 1-10	/sarcoos/pot/1 through /pot/10	float: degrees 0.00-270.00	0
Button 1-4	/sarcoos/btn/1 through /btn/4	int: 0 or 1	1
MPU6050 Roll	/sarcoos/imu/roll	float: degrees	0
MPU6050 Pitch	/sarcoos/imu/pitch	float: degrees	0
MPU6050 Yaw	/sarcoos/imu/yaw	float: degrees (drifts)	0
TCRT5000 Line Sensors	/sarcoos/line/1 through /line/4	int: 0 or 1	0
HC-SR04 Distance	/sarcoos/dist/front	float: centimeters	0
Suit Status LWT	/sarcoos/status/suit	string: online/offline	1

5.4 Rover Output Pipeline (esp32_outputs.cpp)

The output_task runs on the Rover ESP32. It registers per-topic MQTT callbacks that translate incoming telemetry into physical motor commands. The task body itself is nearly empty (a 100 ms delay loop); all actuation logic executes asynchronously in MQTT callback context.

MQTT Topic	Actuator	Mapping Logic
/sarcoos/pot/1	BTS7960 Motor 1	0-270° mapped to -1.0 to +1.0 speed; direction from sign
/sarcoos/pot/2	BTS7960 Motor 2	Same mapping as Motor 1
/sarcoos/pot/3-6	Servo Arms 1-4	0-270° mapped to 0-180° servo angle
/sarcoos/pot/7-10	BTS7960_Pot_Position joints	Direct degree setpoint for closed-loop joints
/sarcoos/btn/1	Emergency Stop (all motors)	data==1: brake all BTS7960 instances
/sarcoos/btn/2	Mode Switch	Toggle between open-loop and closed-loop control
/sarcoos/imu/roll	Stabilization	Compensation input for rover tilt correction

5.5 Position Control Implementation (pot_pos_control.cpp)

The pot_pos_control.cpp file demonstrates the practical integration of BTS7960_Pot_Position as a high-torque joint actuator. Three objects collaborate — BTS7960, Potentiometer, and BTS7960_Pot_Position — to implement a 50 Hz closed-loop position servo using a DC motor with a potentiometer feedback sensor.

```
// Hardware object construction
BTS7960 motor(R_EN, L_EN, R_PWM, L_PWM);
Potentiometer pot(ADC_CHANNEL_6, ADC_ATTEN_DB_11, 270);
BTS7960_Pot_Position joint(motor, pot, KP=3.5f, KI=0.05f, KD=0.20f);

// FreeRTOS control task
static void pos_control_task(void* arg) {
    BTS7960_Pot_Position* joint = (BTS7960_Pot_Position*)arg;
    TickType_t last = xTaskGetTickCount();
    while (true) {
        joint->update(0.020f);           // 20 ms timestep
        vTaskDelayUntil(&last, pdMS_TO_TICKS(20)); // Precise 50 Hz
    }
}
```

```
// MQTT target command
mqtt.on_topic("/sarcoos/pot/7", [&joint](std::string data) {
    float target = std::stof(data);
    joint.setAngle(target);
    joint.reset();    // Reset PID integral on new command
});
```

Chapter 6: Custom PCB Hardware Design

6.1 Design Overview

The SarcoOS custom PCB was designed to consolidate the ESP32 microcontroller, motor driver connections, power management, and I2C/UART peripheral headers onto a single, professionally routed 2-layer PCB. The design prioritizes signal integrity, EMI mitigation, and mechanical robustness for the demanding environment of a mobile robotic platform.

6.2 Microcontroller: ESP32 DevKitC-32E

The ESP32 DevKitC-32E was selected as the computational core of the SarcoOS PCB. It features the ESP32-WROOM-32E module with 4 MB SPI flash, 520 KB SRAM, integrated 2.4 GHz WiFi and Bluetooth, 34 programmable GPIO pins, 4 hardware SPI interfaces, 2 I2C buses, 3 UART ports, and 16 PWM channels (LEDC).

The DevKitC form factor provides USB-to-UART programming via an onboard CP2102 bridge, making in-circuit firmware updates straightforward without requiring external programming hardware. The BOOT and EN buttons on the DevKitC are accessible on the PCB for manual firmware entry and reset.

6.3 Power Distribution System

6.3.1 24V Main Supply

The primary power rail is 24V DC, sourced from a lithium battery pack and delivered to the BTS7960 H-bridge drivers and DC motors. The 24V rail bypasses the ESP32 entirely; the ESP32 and its associated 3.3V logic circuits are powered from the regulated 5V output of the DCDC converter.

6.3.2 DCDC Buck Converter

A synchronous step-down (buck) converter reduces the 24V main supply to a stable 5V output for the ESP32 DevKitC, the logic-level enable pins of the motor drivers, and the PCA9685 servo controller. The converter was selected for its wide input voltage range (8-36V), high efficiency (>90% at full load), and integrated over-current and thermal protection. Large filter capacitors on both input and output suppress switching noise from entering the 3.3V logic domain.

6.3.3 3.3V LDO Regulation

The ESP32's onboard 3.3V LDO (powered from the 5V rail via the USB/VIN pin of the DevKitC) supplies the ADC reference voltage and all GPIO logic levels. Potentiometer voltage dividers are

connected between 3.3V and GND, ensuring the ADC input voltage stays within the ADC1 channel's 0-3.3V range when ADC_ATTEN_DB_11 is selected.

6.4 Ground Strategy: Dual-Layer Ground Plane

Both PCB layers are largely dedicated to copper ground fills. The top layer ground plane underlies all digital signal traces; the bottom layer provides the return path for high-current motor driver circuits. The two ground planes are connected through multiple vias at strategic locations to minimize ground impedance.

The ground plane strategy provides three key benefits: first, it reduces the impedance of return current paths, minimizing voltage drops that would corrupt ADC readings; second, it acts as a Faraday shield that reduces radiated EMI from the switching motor drivers; third, it improves heat dissipation from power components.

6.5 Complete Pin Assignment Table

Signal	ESP32 GPIO	Connected To	Notes
STEP1	IO25	TMC2209 #1 STEP pin	Step pulse output for stepper 1
DIR1	IO26	TMC2209 #1 DIR pin	Direction control for stepper 1
STEP2	IO35	TMC2209 #2 STEP pin	Step pulse output for stepper 2
DIR2	IO12	TMC2209 #2 DIR pin	Direction control for stepper 2
PWML1	IO27	BTS7960 #1 L_PWM	LEDC Channel 0 — Motor 1 reverse
PWMR1	IO14	BTS7960 #1 R_PWM	LEDC Channel 1 — Motor 1 forward
R_EN1/L_EN1	Dedicated GPIO	BTS7960 #1 Enable pins	Enable active HIGH
PWML2	IO32	BTS7960 #2 L_PWM	LEDC Channel 2 — Motor 2 reverse
PWMR2	IO33	BTS7960 #2 R_PWM	LEDC Channel 3 — Motor 2 forward

Signal	ESP32 GPIO	Connected To	Notes
R_EN2/L_EN2	Dedicated GPIO	BTS7960 #2 Enable pins	Enable active HIGH
SDA	IO21	MPU6050 & PCA9685 SDA	I2C_NUM_0 data line
SCL	IO22	MPU6050 & PCA9685 SCL	I2C_NUM_0 clock line (400 kHz)
GPS_TX	IO17	Neo-6M RX	ESP32 transmits to GPS
GPS_RX	IO16	Neo-6M TX	ESP32 receives from GPS
POT1-POT6	ADC1_CH0 to CH5	Potentiometers	GPIO36-GPIO39, GPIO32-GPIO33
ENC_A	IO4	Encoder Channel A	Interrupt-capable, IRAM_ATTR ISR
ENC_B	IO16	Encoder Channel B	Direction read inside ISR

6.6 External Header Connections

6.6.1 Raspberry Pi 4 Header (4-pin)

A 4-pin JST header on the PCB connects the ESP32 UART1 (TX/RX) to the Raspberry Pi 4's UART0 (/dev/ttyAMA0) for high-level command forwarding. This allows ROS 2 nodes on the RPi to send navigation and arm control commands to the ESP32 over a reliable, low-latency serial link as a fallback when WiFi is unavailable.

6.6.2 PCA9685 Header (4-pin)

The PCA9685 is a 16-channel PWM controller that extends the ESP32's 16 LEDC channels to support additional servo outputs for complex robotic arms. It communicates over the shared I2C bus (IO21/IO22) at address 0x40 (default). Each PCA9685 channel generates a 50 Hz PWM signal independently, freeing the ESP32 from managing individual servo PWM timing.

6.6.3 BTS7960 Driver Headers (6-pin x2)

Each BTS7960 H-bridge is connected to the ESP32 PCB via a 6-pin header carrying: R_EN, L_EN, R_PWM, L_PWM, 5V power, and GND. The BTS7960 driver boards are mounted externally on heat sinks, as the 43A continuous current rating generates significant heat that must be dissipated away from the sensitive analog circuitry on the main PCB.

Chapter 7: Mechanical & 3D Design

7.1 CAD Environment: SolidWorks

All mechanical components of the SarcoOS robotic platform were designed in Dassault Systemes SolidWorks, the industry-standard parametric 3D CAD platform. SolidWorks was selected for its comprehensive assembly management, kinematic motion simulation, and direct export capability to URDF (Unified Robot Description Format) via the `sw_urdf_exporter` plugin — a critical path to the ROS 2 simulation workflow.

The design follows a modular sub-assembly approach: the chassis, each arm segment, gripper mechanism, and sensor mounting are independent sub-assemblies that are mated into the master assembly. This modularity enables independent revision of individual components without invalidating unrelated geometry.

7.2 Actuator Inventory & Rationale

Actuator	Quantity	Specification	Application	Rationale
24V DC Motor	2	24V, high RPM with gearbox	Differential drive drivetrain	High torque for navigation over varied terrain; 24V matches main supply
Feetech High-Torque Servo	2	95 kg-cm holding torque	Heavy-duty arm joints (shoulder)	Standard RC servos insufficient; 95 kg-cm handles arm weight at full extension
Feetech Standard Servo	2	35 kg-cm holding torque	Secondary arm joints (elbow/wrist)	35 kg-cm handles reduced moment arm at distal joints
NEMA 23 Stepper Motor	2	2.0-3.0 N-m holding torque	Gripper mechanism (open/close)	Precise, repeatable position without encoder; holding torque prevents gripper slip

7.3 Power Architecture

The robotic platform uses a dual-voltage power architecture to serve the diverse requirements of its actuators:

- 24V Main Supply: Powers the two DC drive motors via BTS7960 H-bridges and the two NEMA 23 stepper motors via TMC2209 drivers. The 24V level maximizes motor efficiency and reduces current draw for a given power output.
- 12V Auxiliary Supply: Powers the four Feetech servo motors. A separate 12V supply prevents the large current transients from the servo motors (which can draw 2-4A during rapid motion) from causing voltage droop on the 24V rail.

7.4 Computational Core: Raspberry Pi 4 Model B

The Raspberry Pi 4 Model B serves as the high-level computing platform for the SarcoOS robotic system. It runs Ubuntu 22.04 LTS and ROS 2 Humble, providing the processing power required for SLAM computations, camera image processing, and ROS 2 navigation stack execution. The RPi4 communicates with the ESP32 over both WiFi (MQTT) and UART, depending on the task.

Specification	Value
CPU	Broadcom BCM2711, quad-core Cortex-A72 @ 1.8 GHz
RAM	4 GB LPDDR4-3200
Storage	32 GB microSD (OS) + USB SSD (ROS 2 maps)
Connectivity	Gigabit Ethernet, WiFi 802.11ac, Bluetooth 5.0
Camera Interface	MIPI CSI-2 for ROS 2 camera node
OS	Ubuntu 22.04 LTS (64-bit)
ROS Version	ROS 2 Humble Hawksbill

7.5 TMC2209 Stepper Motor Driver

7.5.1 StealthChop Technology

The TMC2209 driver from Trinamic (acquired by Analog Devices) implements the StealthChop2 current control algorithm, which enables near-silent stepper motor operation at low and medium speeds. This is achieved by using a voltage-mode driver model that continuously adapts coil excitation to match back-EMF, eliminating the audible switching noise typical of traditional stepper drivers.

7.5.2 STEP/DIR Interface

The STEP/DIR interface is the simplest and most universal stepper motor control protocol. Each rising edge on the STEP pin advances the motor by one microstep; the DIR pin determines the stepping direction. The TMC2209 supports up to 256 microsteps per full step, providing up to 51,200 microstep positions per revolution on a 200-step (1.8°/step) NEMA 23 motor.

In SarcoOS, the stepper motors drive the gripper mechanism. The step generation is handled directly by ESP32 GPIO toggling from the application task, with step pulse timing calculated from the desired velocity and microstep resolution:

```
// Gripper step generation
const int MICROSTEPS = 16;           // TMC2209 configured for 16x microstepping
const int STEPS_PER_REV = 200;       // NEMA 23 base resolution
const float LEAD_MM_PER_REV = 8.0;   // Lead screw pitch
// Calculate step pulse period for target velocity
float steps_per_mm = (STEPS_PER_REV * MICROSTEPS) / LEAD_MM_PER_REV;
float pulse_period_us = 1000000.0f / (velocity_mm_s * steps_per_mm);
```

Chapter 8: Teleoperation Wearable Suit

8.1 Concept and Design Philosophy

The SarcoOS teleoperation suit is a wearable human-machine interface (HMI) that enables an operator to control the robot through natural body movements. Rather than requiring the operator to manipulate joysticks or touchscreen controls, the suit maps the operator's own joint angles directly to the corresponding robot joint angles, creating an intuitive 1:1 kinematic correspondence between human and robot.

This direct kinematic mapping reduces the cognitive load on the operator compared to traditional HMI approaches and enables faster, more natural robot manipulation — particularly for tasks requiring fine motor coordination such as grasping fragile objects.

8.2 Hardware Architecture

8.2.1 Microcontroller

The Suit uses an ESP32 DevKitC module — the same microcontroller family as the Rover — for firmware code reuse and a familiar development environment. The ESP32's dual-core Xtensa LX6 processor provides sufficient computational headroom to run the sensor polling, MQTT publishing, and FreeRTOS scheduling simultaneously with a comfortable margin.

8.2.2 Joint Angle Sensing: 8x Potentiometers

Eight 10 kΩ rotary potentiometers are mounted at the articulated joints of the suit exoskeleton. Each potentiometer is mechanically coupled to a joint rotation axis, converting the physical joint angle into a proportional voltage that the ESP32 ADC measures. The potentiometers are wired as voltage dividers between 3.3V and GND:

$$V_{out} = 3.3V \times (R_{wiper} / R_{total})$$

As the joint rotates from 0° to its full range (e.g., 270°), V_{out} sweeps linearly from 0V to 3.3V. The Potentiometer driver's `readAngle()` method converts this voltage to an angle in degrees.

8.2.3 Joint Mapping Table

Potentiometer	Joint	MQTT Topic	Range
POT 1	Right Shoulder Flexion/Extension	/sarcoos/pot/1	0-180°

Potentiometer	Joint	MQTT Topic	Range
POT 2	Right Elbow Flexion/Extension	/sarcoos/pot/2	0-150°
POT 3	Left Shoulder Flexion/Extension	/sarcoos/pot/3	0-180°
POT 4	Left Elbow Flexion/Extension	/sarcoos/pot/4	0-150°
POT 5	Right Wrist Rotation	/sarcoos/pot/5	0-270°
POT 6	Left Wrist Rotation	/sarcoos/pot/6	0-270°
POT 7	Torso Rotation	/sarcoos/pot/7	0-180°
POT 8	Neck Pan	/sarcoos/pot/8	0-180°

8.2.4 Gripper Button

A momentary push button mounted on the suit's right glove provides binary gripper control. A single press toggles the gripper between open and closed states. The button is published to `/sarcoos/btn/1` with QoS 1, ensuring the toggle command is delivered reliably even under temporary WiFi congestion. The Rover's output task maintains a `gripper_open` boolean state variable that flips on each rising edge received from this topic.

8.3 Communication Architecture

The Suit publishes all sensor data to the MQTT broker at 50 Hz. The broker (Mosquitto, running on the Raspberry Pi 4 or a dedicated server) routes these messages to all subscribers: the Rover ESP32 for direct motion execution, and the Node-RED HMI for visualization.

The end-to-end latency from human joint movement to robot joint response consists of: ADC sampling (negligible), MQTT publish over WiFi (approximately 5-15 ms on a local network), MQTT broker routing (1-2 ms), MQTT receive on Rover (negligible), and PID control response (one 20 ms control tick). Total round-trip latency is typically 25-40 ms — imperceptible to a human operator.

8.4 Last Will and Testament (LWT)

The Suit MQTT client registers a Last Will and Testament message on the `/sarcoos/status/suit` topic. If the Suit disconnects unexpectedly (power loss, WiFi outage), the broker automatically publishes the LWT message with payload `offline` to all subscribers. The Rover's output task monitors this topic and executes a safe shutdown sequence (brake all motors, hold current positions) when the `offline` status is received.

```
// LWT configuration in Suit's MQTTClient
esp_mqtt_client_config_t mqtt_cfg = {
    .uri = BROKER_URL,
    .lwt_topic = "/sarcoos/status/suit",
    .lwt_msg = "offline",
    .lwt_qos = 1,
    .lwt_retain = 1    // Retained: new subscribers see last known state
};
```

Chapter 9: ROS 2, Simulation & SLAM Navigation

9.1 ROS 2 Architecture Overview

The Robot Operating System 2 (ROS 2) is the middleware layer running on the Raspberry Pi 4 that coordinates high-level robotic functions: SLAM mapping, localization, path planning, and sensor fusion. ROS 2 uses a Distributed Data System (DDS) publish/subscribe communication model, where nodes communicate by publishing and subscribing to typed topics without point-to-point connections.

In SarcoOS, ROS 2 complements the ESP32 MQTT layer: the ESP32 handles real-time, low-level motor control with hard timing requirements; ROS 2 handles computationally intensive, high-level tasks like SLAM and navigation that benefit from the RPi4's general-purpose computing capabilities.

9.2 URDF Export from SolidWorks

9.2.1 SolidWorks to URDF Workflow

The SarcoOS robot's mechanical model, designed in SolidWorks, must be converted to URDF (Unified Robot Description Format) for use in Gazebo simulation and the ROS 2 `robot_state_publisher` node. The `sw_urdf_exporter` plugin for SolidWorks automates this conversion:

21. In SolidWorks, assign coordinate frames (reference axes) to each joint in the robot assembly using the SolidWorks Coordinate System feature.
22. Define the kinematic tree: identify each link (rigid body) and each joint (rotation/translation axis) in the assembly hierarchy.
23. Run the URDF exporter: SolidWorks File > Export > Save as URDF. The plugin generates the URDF XML, STL mesh files for each link, and a package directory structure.
24. Review and refine the generated URDF: check joint axis directions, inertial properties (mass, center of mass, inertia tensor), and visual/collision mesh assignments.

9.2.2 URDF Structure

```
<!-- Simplified URDF for SarcoOS chassis -->
<robot name="sarcoos">
  <link name="base_link">
    <visual>
      <geometry><mesh filename="package://sarcoos/meshes/chassis.stl"/></geometry>
    </visual>
    <inertial>
      <mass value="5.0"/>
      <inertia ixx="0.1" ixy="0" ixz="0" iyy="0.1" iyz="0" izz="0.2"/>
    </inertial>
  </link>
```

```
<joint name="left_wheel_joint" type="continuous">
  <parent link="base_link"/>
  <child link="left_wheel"/>
  <axis xyz="0 1 0"/>
  <origin xyz="-0.15 0.2 0"/>
</joint>
</robot>
```

9.3 Gazebo Simulation

9.3.1 Simulation Environment

Gazebo is the physics-based 3D simulation environment used to validate the SarcoOS robot's kinematics and control algorithms before hardware deployment. The SarcoOS Gazebo model includes the URDF robot description plus a set of Gazebo plugins that simulate the physical behavior of sensors and actuators.

9.3.2 Required Gazebo Plugins

Plugin	Purpose	ROS 2 Topic Published
libgazebo_ros_diff_drive.so	Simulates differential drive dynamics from cmd_vel commands	/cmd_vel (input), /odom (output)
libgazebo_ros_imu_sensor.so	Publishes simulated IMU data from physics engine	/imu/data
libgazebo_ros_joint_state_publisher.so	Publishes joint angles for all robot joints	/joint_states
libgazebo_ros_camera.so	Simulates RGB camera and publishes image stream	/camera/image_raw
libgazebo_ros_ray_sensor.so	Simulates LIDAR for SLAM testing	/scan

9.3.3 Launch File

```
# sarcoos_gazebo.launch.py
import os
from launch import LaunchDescription
from launch.actions import IncludeLaunchDescription
from launch_ros.actions import Node
from ament_index_python.packages import get_package_share_directory

def generate_launch_description():
    pkg_path = get_package_share_directory('sarcoos_description')
```

```
urdf_file = os.path.join(pkg_path, 'urdf', 'sarcoos.urdf')
with open(urdf_file, 'r') as f:
    robot_description = f.read()
return LaunchDescription([
    Node(package='robot_state_publisher',
        executable='robot_state_publisher',
        parameters=[{'robot_description': robot_description}]),
    Node(package='gazebo_ros',
        executable='spawn_entity.py',
        arguments=['-topic', 'robot_description', '-entity', 'sarcoos']),
])
```

9.4 SLAM: Simultaneous Localization and Mapping

9.4.1 SLAM Concept

SLAM is the computational process by which a robot simultaneously builds a map of an unknown environment (Mapping) and estimates its own position within that map (Localization). The chicken-and-egg nature of this problem — you need a map to localize, and you need to know your location to build an accurate map — is what makes SLAM a challenging and extensively researched problem.

9.4.2 SLAM Toolkit: slam_toolbox

SarcoOS uses the `slam_toolbox` package (the standard ROS 2 SLAM implementation) in online asynchronous mode. `slam_toolbox` implements Graph-Based SLAM using GTSAM (Georgia Tech Smoothing and Mapping) as the backend optimizer.

The SLAM process works as follows:

25. The robot moves through the environment, collecting LIDAR scan data (/scan topic) and odometry data (/odom from the differential drive plugin or wheel encoder integration).
26. `slam_toolbox` correlates each new LIDAR scan with the existing map using scan matching (the Correlative Scan Matcher or ICP algorithm).
27. The robot's estimated pose is refined by minimizing the mismatch between the scan and the expected map appearance at the estimated location.
28. Pose constraints from odometry and scan matching are added as edges in a factor graph.
29. Periodically, GTSAM solves the factor graph optimization to reduce accumulated drift.
30. When the robot revisits a previously mapped area (loop closure detection), a new constraint is added between the current and historical poses, dramatically reducing map drift.

9.4.3 SLAM Mathematics: Scan Matching

The core of the scan matching algorithm minimizes the following objective function over the robot pose correction (Δx , Δy , $\Delta \theta$):

```
minimize: sum_i || map_point_i - transform(scan_point_i, Δx, Δy, Δθ) ||^2
```

Where transform applies the 2D rigid body transformation to each scan point. This least-squares problem is solved iteratively (typically Gauss-Newton or Levenberg-Marquardt) until convergence. The ICP (Iterative Closest Point) algorithm solves this by alternating between finding the closest map point for each scan point and optimizing the transformation.

9.4.4 SLAM Launch File

```
# sarcoos_slam.launch.py
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    slam_params = {
        'use_sim_time': True,
        'slam_toolbox': {
            'odom_frame': 'odom',
            'map_frame': 'map',
            'base_frame': 'base_link',
            'scan_topic': '/scan',
            'mode': 'mapping',
            'solver_plugin': 'solver_plugins::CeresSolver',
            'minimum_travel_distance': 0.5,
            'minimum_travel_heading': 0.5,
        }
    }
    return LaunchDescription([
        Node(package='slam_toolbox',
            executable='async_slam_toolbox_node',
            parameters=[slam_params],
            remappings=[('/scan', '/sarcoos/scan')]),
    ])
```

9.5 Sensor Fusion

9.5.1 IMU + Odometry Fusion with robot_localization

The robot_localization package provides an Extended Kalman Filter (EKF) that fuses multiple state estimates into a single, consistent pose estimate. In SarcoOS, it fuses:

- Wheel odometry (/odom): Low-noise velocity estimates, but accumulates drift from wheel slip over time.
- IMU data (/imu/data): Provides accurate angular velocity and linear acceleration. Drift-free for short-duration orientation estimation.

The EKF state vector is $[x, y, z, \text{roll}, \text{pitch}, \text{yaw}, v_x, v_y, v_z, v_{\text{roll}}, v_{\text{pitch}}, v_{\text{yaw}}, a_x, a_y, a_z]$. Each sensor update is weighted by its covariance matrix, with lower-covariance (more reliable) measurements receiving higher weight.

9.6 Navigation: Nav2 Stack

The Nav2 (Navigation 2) framework provides autonomous navigation capabilities including path planning, obstacle avoidance, and goal-directed motion. Key components in the SarcoOS Nav2 configuration:

- BT Navigator: A Behavior Tree-based navigator that orchestrates the navigation pipeline. It sequences: goal received -> compute path -> follow path -> arrived.
- Controller Server (DWB Local Planner): Generates `cmd_vel` commands to follow the global path while avoiding local obstacles detected in real-time.
- Planner Server (NavFn): Computes the global shortest path from the robot's current position to the goal using Dijkstra's algorithm on the occupancy grid map.
- Costmap 2D: Maintains both a global costmap (based on the SLAM map) and a local costmap (based on recent sensor data) that encode obstacle distances for safe path planning.

```
# Nav2 launch configuration
from launch_ros.actions import Node
nav2_params = {
    'controller_server': {
        'controller_plugins': ['FollowPath'],
        'FollowPath': {
            'plugin': 'dwb_core::DWBLocalPlanner',
            'max_vel_x': 0.5,
            'min_vel_x': -0.25,
            'max_vel_theta': 1.0,
            'min_vel_theta': -1.0,
        }
    }
}
```

Chapter 10: Performance, Safety & System Validation

10.1 Real-Time Performance Metrics

Metric	Target	Achieved	Method
Sensor poll rate	50 Hz	50 Hz $\pm$ 0.1 Hz	vTaskDelayUntil() with FreeRTOS tick
MQTT end-to-end latency	< 50 ms	25-40 ms (LAN)	Measured via oscilloscope trigger on pot move + motor response
PID control period jitter	< 1 ms	< 0.5 ms	Task priority 7 preempts all lower-priority operations
ADC voltage accuracy	$\pm$ 50 mV	$\pm$ 30 mV	eFuse calibration with adc_cal_line_fitting
Position control accuracy	$\pm$ 5°	$\pm$ 2°	PID tolerance threshold; measured over 50 positioning cycles

10.2 Safety Architecture

10.2.1 Emergency Stop Chain

The emergency stop chain in SarcoOS is designed with defense-in-depth: multiple independent mechanisms can each bring the system to a safe state, ensuring that a failure in any single mechanism does not prevent safe shutdown.

- 31. Hardware: BTS7960 enable pins pulled LOW by hardware pull-down resistors. Power loss = motor disable.
- 32. MQTT: /sarcoos/btn/1 published with QoS 1. Broker retains and re-delivers on reconnect.
- 33. LWT: Suit MQTT LWT publishes offline to /sarcoos/status/suit on unexpected disconnect. Rover enters safe hold state.
- 34. Watchdog: ESP32 hardware watchdog timer triggers ESP.restart() if any FreeRTOS task starves the watchdog feed task.

10.2.2 Memory Safety

Stack overflow detection is enabled for all FreeRTOS tasks via `configCHECK_FOR_STACK_OVERFLOW = 2` in `FreeRTOSConfig.h`. This causes an immediate assertion failure and system halt if any task's stack pointer crosses its allocated boundary, preventing silent memory corruption.

10.3 Memory Budget Analysis

Component	Stack / Heap Allocation	Notes
input_task	4,096 bytes stack	Sensor polling + MQTT publish
output_task	4,096 bytes stack	MQTT callback + motor actuation
pos_control_task	2,048 bytes stack	PID loop only; minimal stack needed
MQTTClient internal	~8 KB heap	RX/TX buffers + esp_mqtt_client internals
WiFiManager	~16 KB heap	TCP/IP stack via lwIP
NVS partition	16 KB flash	Default nvs partition; expandable
Total heap (typical)	~40 KB of 320 KB	Safe margin; 280 KB available for expansion

10.4 MQTT Reliability Matrix

Failure Scenario	Mitigation	Recovery Time
WiFi dropout	WiFiManager auto-reconnects with exponential backoff up to max_retries	5-30 seconds typical
MQTT broker disconnect	MQTTClient clean_session=false; QoS 1 emergency commands redelivered on reconnect	Immediate on reconnect
Motor stall / overcurrent	BTS7960 disable() de-energizes driver ICs; application watchdog detects stall	Instantaneous
PID integral windup on joint block	Anti-windup clamp prevents integral explosion; bounded output on unblock	One PID control tick

Failure Scenario	Mitigation	Recovery Time
ADC noise spikes from motor EMI	readAveraged(N) smooths N samples; increasing N trades latency for rejection	$N \times \text{ADC sample time}$
ESP32 task crash / exception	ESP32 core dump to flash; hardware watchdog triggers reboot	< 3 seconds reboot

10.5 Validation Procedures

10.5.1 Driver-Level Unit Tests

Each HAL driver is validated independently before system integration:

- **BTS7960:** Forward/backward at 0%, 50%, 100% duty cycle; brake and stop response time measurement; enable/disable sequencing verification.
- **Potentiometer:** Linearity check at 10° intervals across full 270° range; ADC noise floor measurement with and without motor power; calibration accuracy vs. reference DMM.
- **MPU6050:** Static calibration verification (zero offsets at rest); complementary filter step response test with known rotation angle.
- **PID:** Unit step response test with simulated plant (PT1 model); anti-windup verification by blocking motor and measuring integral bound.

10.5.2 System Integration Tests

Full system tests validate end-to-end behavior:

- **Latency test:** Measure MQTT publish to motor response time using oscilloscope triggers on GPIO events.
- **50 Hz consistency test:** Monitor MQTT topic publish timestamps over 60 seconds; verify period stability within ± 1 ms.
- **Emergency stop test:** Simulate WiFi dropout and verify Rover enters safe hold state via LWT mechanism within 2 seconds.
- **Position control accuracy:** Command 20 different setpoints across the 0-270° range; measure steady-state error with digital protractor.

10.6 Future Development Roadmap

Enhancement	Priority	Description
Force/Torque Sensing	High	Add load cells to gripper fingers for haptic feedback to the operator
Vision-Based Object Detection	High	Integrate YOLO inference on RPi 4 for autonomous object detection in teleoperation
ROS 2 MQTT Bridge	Medium	Implement a ROS 2 node that bridges the MQTT topics to ROS 2 topics for unified Nav2 integration
Over-the-Air (OTA) Updates	Medium	Implement ESP-IDF OTA to enable firmware updates via WiFi without physical USB access
Battery Management System	Medium	Add INA226 power monitors on each supply rail for real-time power consumption telemetry
IMU Magnetometer Fusion	Low	Add HMC5883L magnetometer to eliminate IMU yaw drift for long-duration operations

10.7 Conclusion

The SarcoOS project successfully demonstrates a complete, production-quality RTOS-based robotic control ecosystem on the ESP32 platform. The three-tier layered architecture (Application / HAL / MCAL) provides a clean separation of concerns that enables independent development and testing of individual subsystems. The MQTT-based communication layer provides the low-latency, asynchronous data transport required for real-time teleoperation, while FreeRTOS ensures deterministic timing for all control loops.

The integration with ROS 2, Gazebo simulation, and the SLAM/Navigation stack demonstrates the scalability of the architecture: the same robotic platform that can be controlled manually via the wearable suit can also operate autonomously using the Nav2 navigation stack, guided by a map built through slam_toolbox SLAM.

The SarcoOS codebase represents a significant contribution to the academic and open-source robotics community, providing a well-documented, modular reference implementation for ESP32-based robotic control that can be adapted for a wide range of teleoperation and autonomous mobile robot applications.

— *End of SarcoOS Technical Manual v1.0* —

Helwan International Technology University | Academic Year 2024/2025